

Relatório Técnico Final — BluaDiagnostics Sprint 4

Evolução de Engenharia e Refatoração de Arquitetura

Resumo da Evolução Arquitetural

Este relatório formaliza a migração estrutural do ecossistema BluaDiagnostics na Sprint 4. O objetivo principal foi desmobilizar a Prova de Conceito (POC) original, que centralizava decisões em um único prompt massivo e linear, para implementar um sistema **Multi-Agente Orquestrado via LangGraph**. Essa nova arquitetura garante isolamento absoluto de contextos, validações de segurança automatizadas e ferramentas desacopladas.

Matriz de Mudanças Estruturais

Componente Antigo	Nova Estrutura (Sprint 4)	Impacto e Justificativa Técnico
<code>workflow.py</code> e prompt único	<code>state.py</code> , <code>routing.py</code> , <code>builder.py</code>	Isolamento da máquina de estados, regras dinâmicas de decisão e compilação limpa do grafo.
<code>system_prompt.md</code> unificado	Múltiplos <code>.md</code> em <code>/prompts</code>	Redução no consumo de tokens, mitigação de alucinações e especialização por domínio.
<code>tools_spec.json</code>	Ferramentas modulares em <code>/tools</code>	Eliminação de esquemas estáticos JSON. Geração automática via assinaturas do LangChain.
<code>chroma_setup.py</code> monolítico	Módulos especializados em <code>/rag</code>	Estratégia de chunking encapsulada, persistência local estável e recuperação semântica independente.
Instruções textuais de segurança	Módulos em <code>/safety</code>	Validação determinística interceptando requisições antes e depois de acionar o LLM.

Decisões Arquiteturais e Trade-offs

A migração exigiu adaptações críticas para equilibrar as limitações de hardware local com a necessidade de um fluxo conversacional seguro.

1. Troca de LLM Base (Llama 3.1 8B vs. Qwen 2.5 14B)

- **O Problema:** O Llama 3.1 (8B) falhava nas regras de roteamento e alucinava o uso de ferramentas. O ideal seria um modelo de 70B, mas o hardware atual não suporta os ~40GB livres necessários.

- **A Solução:** Migração para o Qwen 2.5 (14B). Ele maximiza o uso da GPU e utiliza RAM excedente, oferecendo altíssima obediência a prompts e redução de erros, com um pequeno trade-off na velocidade de geração. O impacto no código foi nulo (via LangChain).

2. Orquestração Roteamento e Contexto (LangGraph)

- **Structured Output vs. Texto Livre:** O Supervisor agora é forçado a preencher um schema Pydantic rigoroso (JSON) em vez de texto livre. Isso garante roteamento 100% determinístico, eliminando o "tagarelistismo". Em caso de falha de formatação do LLM, um bloco *try/except* redireciona para a triagem.
- **Estado Único vs. Agentes Concorrentes:** Optou-se pelo repasse de estado (`state["messages"]`) em vez de rodar agentes em paralelo. Isso economiza VRAM e evita condições de corrida. O trade-off é o crescimento do contexto a longo prazo, exigindo futuramente um nó de sumarização.

3. Especialização dos Agentes Internos

- **Supervisor:** Classificador determinístico de intenções.
- **Agente de Triagem:** Especializado no acolhimento de queixas, coleta de sintomas e leitura de sinais de wearables.
- **Agente de Prescrição:** Restrito a consultas de bulas e interações, blindado contra a criação/alteração de receitas.
- **Agente de Escalonamento:** Acionado em cenários limítrofes, aplicando protocolos de urgência.

4. Gestão de Ferramentas (Tool Hiding)

- **A Abordagem:** Para evitar que LLMs menores inventem parâmetros (alucinação de dados) para usar ferramentas rapidamente, adotou-se o "Tool Hiding".
- **O Resultado:** As ferramentas ficam ocultas nas primeiras interações (`if msgs_validas <= 2`), forçando a IA a atuar como um chatbot de coleta de dados real. O trade-off é a maior complexidade no código e a impossibilidade de a IA usar ferramentas no primeiro prompt, mitigado por docstrings rigorosas (`@tool`) em caixa alta.

Camada de Segurança e "Escudo de Entrada"

A segurança foi dividida entre proteções internas do LangGraph e um escudo pré-inferência para otimizar o tempo de resposta e poupar a GPU.

- **Escudo de Entrada (Python/Regex):** Trava de segurança no Streamlit que atua antes do LangGraph. Funções de `verificar_toxidade` e `validar_escopo_medico` leem o texto e bloqueiam xingamentos ou assuntos fora de escopo (ex: receitas de bolo) com latência zero. Isso resolveu o problema da IA perder tempo sendo "educada" com usuários hostis, embora dependa de uma lista de Regex estática.
- **Deteção de Jailbreak:** Filtragem avançada contra ataques de injeção de prompt e tentativas de exposição do sistema.
- **Filtro de Red Flags Clínicas:** Varredura imediata de termos graves (infarto, ideação suicida) provocando escalada automática à equipe médica real.

Resultados de Avaliação (Evals)

A suíte de testes dinâmicos (`run_evals.py`) consumiu o mapeamento `scenarios.json` para rastrear latência, acurácia e custo simulado, gerando os seguintes resultados na Sprint 4:

13.65s

Tempo Médio de Resposta

\$0.000014

Custo Médio por Chamada

\$0.000251

Custo Total Simulado

100.0%

Taxa de Escalada Correta

Acurácia de Roteamento por Categoria:

- **Happy Path:** 100.0%
- **Jailbreak:** 100.0%
- **Fora de Escopo:** 100.0%
- **Bloqueio de Agendamento:** 100.0%
- **Alerta de Prescrição:** 100.0%
- **Moderação de Conteúdo:** 100.0%
- **Red Flag:** 75.0%

Roadmap

O planejamento de execução do sistema foi estruturado em 8 fases incrementais:

Fase 0 — Fundação da Arquitetura

Refatoração de pastas (agentes, graph, safety, prompts), centralização de configurações e organização de schemas Pydantic.

Fase 1 — Arquitetura Multiagente

Implementação do Supervisor, agentes de Triagem, Prescrição e Escalada, e integração completa do LangGraph com estado compartilhado.

Fase 2 — Pipeline RAG

Implementação de chunking semântico, embeddings com Ollama, Ingestão ChromaDB e integração do retriever na interface de chat.

Fase 3 — Segurança e Guardrails

Escudo contra red flags, moderação de conteúdo, detecção de jailbreak e validação de escopo.

Fase 4 — Function Calling e Tools

Desenvolvimento de ferramentas para pacientes, agendamentos, prescrição e emergências com validação em Pydantic.

Fase 5 — Evals Automatizados

Criação de datasets, suíte de avaliação com métricas de acurácia, latência e custo financeiro.

Fase 6 — Observabilidade

Integração com LangSmith, logs estruturados e registro das trajetórias de ferramentas e documentos.

Fase 7 — Interface e Entrega

Finalização do frontend Streamlit, documentação e gravação de vídeo demonstrativo.

Integrantes

- Julia Yamazaki — RM: 568438
- Roberto Flaquer — RM: 567348
- Bryan de Almeida — RM: 568081
- Daniel Silva — RM: 567894
- Guilherme Blanco — RM: 566746
- Jessica Guiot — RM: 568024